INITIATION PYTHON · JOUR 1 SUR 5 · LUNDI 2 NOVEMBRE

Lundi · Premiers pas en Python

Depuis trois semaines, vous écrivez des algorithmes en bash. Cette semaine, on passe à Python. Les idées ne changent pas : variables, conditions, boucles, fonctions. Ce qui change, c'est l'écriture. Plus de `$`, plus de `(( ))`, plus de `fi` ni de `done`. Python se lit presque comme de l'anglais, et c'est l'un des langages les plus utilisés au monde, du petit script au site web en passant par la data.

Jour	Programme
Lundi	VS Code, variables, calculs, saisie au clavier, conditions
Mardi	boucles, listes, chaînes de caractères, fonctions
Mercredi	TP : le jeu du pendu, en Python, publié sur GitHub
Jeudi	dictionnaires, modules, fichiers, erreurs
Vendredi	algorithmes classiques, récursivité, organiser son code

Tout au long de la semaine, les encadrés **« En bash »** font le lien avec ce que vous connaissez déjà.

1. Installer VS Code

Jusqu'ici, vous écriviez vos scripts avec nano. Pour Python, on passe à **VS Code**, l'éditeur de code le plus utilisé : coloration, complétion, erreurs soulignées avant même de lancer le programme, et un terminal intégré.

Installer, dans le terminal de votre VM Ubuntu :

```
sudo snap install code --classic
code --version
```

La deuxième commande doit afficher un numéro de version. Si `snap` n'est pas disponible, téléchargez le fichier `.deb` sur code.visualstudio.com, puis `sudo apt install ./code_*.deb` dans le dossier de téléchargement.

Vérifier Python. Il est déjà installé sur Ubuntu :

Dans le terminal

```
$ python3 --version
Python 3.12.3
```

Sur Ubuntu, la commande s'appelle `python3`, pas `python`. Si vous tapez `python`, Ubuntu répond que la commande n'existe pas.

Créer son dossier de la semaine et l'ouvrir dans VS Code :

```
mkdir ~/python-semaine
cd ~/python-semaine
code .
```

Le point veut dire « le dossier où je suis ». Au premier lancement, VS Code demande si vous faites confiance aux auteurs du dossier : c'est vous, répondez **Yes, I trust the authors**.

Installer l'extension Python. Dans la barre de gauche, l'icône des quatre carrés (**Extensions**, ou `Ctrl+Shift+X`). Cherchez « Python » et installez celle de **Microsoft** (plusieurs millions de téléchargements, badge bleu).

Trois réglages qui évitent les ennuis :

1. **File**, **Auto Save** : VS Code enregistre tout seul. Sinon, vous lancerez l'ancienne version du fichier sans comprendre pourquoi rien ne change.
2. **Terminal**, **New Terminal** : un terminal s'ouvre en bas, déjà placé dans votre dossier. C'est là qu'on lance les programmes.
3. En bas à droite, VS Code affiche la version de Python utilisée. S'il demande d'en choisir une (**Select Interpreter**), prenez `Python 3.12` dans `/usr/bin/python3`.

2. L'interpréteur interactif

Tapez `python3` tout seul dans le terminal. L'invite devient `>>>` : Python attend une instruction, l'exécute, et affiche le résultat tout de suite. C'est parfait pour tester une idée.

Dans l'interpréteur Python

```
$ python3
>>> 2 + 3
5
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> 2 ** 10
1024
>>> (2 + 3) * 4
20
>>> "Bonjour" + " Alice"
'Bonjour Alice'
>>> len("Bonjour")
7
```

Pour en sortir : `exit()`, ou `Ctrl+D`.

Opération	Python	Exemple	Résultat
addition, soustraction, multiplication	`+` `-` `*`	`6 * 7`	`42`
division	`/`	`7 / 2`	`3.5`, toujours un nombre à virgule
division entière	`//`	`7 // 2`	`3`
reste (modulo)	`%`	`7 % 2`	`1`
puissance	`**`	`2 ** 10`	`1024`

> **En bash**, `$(( 7 / 2 ))` donnait 3 : bash ne connaît que les nombres entiers. En Python, `/` fait une vraie division, et `//` la division entière de bash.

3. Votre premier script

Dans VS Code, **File**, **New File**, nommez-le `bonjour.py`. L'extension `.py` dit à VS Code que c'est du Python.

bonjour.py

```python
# Mon premier programme Python
print("Bonjour !")
print("Je m'appelle Alice et j'apprends Python.")
print(6 * 7)
```

Dans le terminal

```
$ python3 bonjour.py
Bonjour !
Je m'appelle Alice et j'apprends Python.
42
```

Pour le lancer, dans le terminal de VS Code : `python3 bonjour.py`. Vous pouvez aussi cliquer sur le triangle ▶ en haut à droite de l'éditeur, qui tape la commande à votre place.

- `print(...)` affiche ce qu'on lui donne, puis passe à la ligne.
- Le texte s'écrit entre guillemets, doubles ou simples : `"Bonjour"` ou `'Bonjour'`. Les nombres, sans guillemets.
- Ce qui suit `#` est un commentaire, comme en bash.
- Pas besoin de `#!/bin/bash` ni de `chmod +x` : on lance avec `python3 fichier.py`.

4. Les variables

variables.py

```python
prenom = "Alice"
age = 32
taille = 1.68
inscrite = True

print(prenom)
print(age + 1)
print(type(prenom), type(age), type(taille), type(inscrite))
```

Dans le terminal

```
$ python3 variables.py
Alice
33
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

On crée une variable en lui donnant une valeur avec `=`. Pas de `$` pour la lire, et les espaces autour du `=` sont permis (et conseillés). Chaque valeur a un **type** :

Type	Nom en Python	Exemples
nombre entier	`int`	`32`, `-5`, `0`
nombre à virgule	`float`	`1.68`, `3.0` (avec un **point**, pas une virgule)
texte, ou chaîne de caractères	`str`	`"Alice"`, `'42'`
booléen, vrai ou faux	`bool`	`True`, `False` (avec la majuscule)

`"42"` et `42` ne sont pas la même chose : le premier est un texte de deux caractères, le second un nombre. Python ne les mélange pas :

Dans l'interpréteur Python

```
>>> "42" + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> int("42") + 1
43
>>> "42" + str(1)
'421'
```

Lisez la dernière ligne de l'erreur : `TypeError`, on ne peut pas coller un `str` et un `int`. Les fonctions `int()`, `float()` et `str()` convertissent d'un type à l'autre.

Nommer ses variables : en minuscules, avec des `_` entre les mots (`note_maximale`, `nombre_de_joueurs`), sans accent, sans espace, sans commencer par un chiffre.

> **En bash** : `prenom="Alice"` sans espace, puis `echo "$prenom"`. En Python : `prenom = "Alice"`, puis `print(prenom)`. Et en Python, une variable qui n'existe pas provoque une erreur, au lieu de valoir une chaîne vide.

5. Afficher proprement : les f-strings

Pour mélanger du texte et des variables, on met un `f` devant les guillemets, et les variables entre accolades :

fstring.py

```python
prenom = "Alice"
age = 32
print(f"{prenom} a {age} ans.")
print(f"Dans 10 ans, elle aura {age + 10} ans.")
print(f"Un tiers : {1 / 3:.2f}")
print("Sans f-string :", prenom, "a", age, "ans.")
```

Dans le terminal

```
$ python3 fstring.py
Alice a 32 ans.
Dans 10 ans, elle aura 42 ans.
Un tiers : 0.33
Sans f-string : Alice a 32 ans.
```

Entre les accolades, on peut mettre n'importe quel calcul. Le `:.2f` arrondit l'affichage à deux chiffres après la virgule. La dernière ligne montre l'autre façon : `print` avec plusieurs valeurs séparées par des virgules, qu'il affiche séparées par un espace.

> **En bash** : `echo "$prenom a $age ans"`. En Python : `print(f"{prenom} a {age} ans")`. Oublier le `f` est l'erreur la plus fréquente : Python affiche alors les accolades telles quelles.

6. Lire au clavier : `input`

age.py

```python
prenom = input("Votre prénom : ")
age = int(input("Votre âge : "))
print(f"Bonjour {prenom}, l'an prochain vous aurez {age + 1} ans.")
```

Pour le premier essai, on tape `32`. Pour le second, `trente-deux` :

Deux essais

```
$ python3 age.py
Votre prénom : Alice
Votre âge : 32
Bonjour Alice, l'an prochain vous aurez 33 ans.
$ python3 age.py
Votre prénom : Alice
Votre âge : trente-deux
Traceback (most recent call last):
  File "/home/alice/python-semaine/age.py", line 2, in <module>
    age = int(input("Votre âge : "))
          ^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'trente-deux'
```

`input("...")` affiche la question, attend que l'utilisateur tape quelque chose et appuie sur Entrée, puis renvoie ce qu'il a tapé. **Toujours sous forme de texte**, même si c'est un nombre : d'où le `int(...)` autour.

Avec « trente-deux », `int()` ne sait pas faire, et Python s'arrête avec un message d'erreur, un *traceback*. Il se lit **de bas en haut** :

1. la dernière ligne dit **quoi** : `ValueError: invalid literal for int() with base 10: 'trente-deux'`, c'est-à-dire « ce texte n'est pas un nombre entier » ;
2. au-dessus, **où** : le fichier, le numéro de ligne (`line 2`), et la ligne elle-même.

Prenez l'habitude de lire ces messages au lieu de les fuir : ils disent presque toujours exactement ce qui ne va pas. Jeudi, on apprendra à les intercepter.

> **En bash** : `read -p "Votre âge : " age`. En Python : `age = int(input("Votre âge : "))`.

7. Les conditions

majeur.py

```python
age = int(input("Votre âge : "))

if age >= 18:
    print("Vous êtes majeur.")
    print("Vous pouvez voter.")
else:
    print("Vous êtes mineur.")
    print(f"Encore {18 - age} an(s) à attendre.")

print("Au revoir.")
```

Trois essais

```
$ python3 majeur.py
Votre âge : 17
Vous êtes mineur.
Encore 1 an(s) à attendre.
Au revoir.
$ python3 majeur.py
Votre âge : 18
Vous êtes majeur.
Vous pouvez voter.
Au revoir.
$ python3 majeur.py
Votre âge : abc
Traceback (most recent call last):
  File "/home/alice/python-semaine/majeur.py", line 1, in <module>
    age = int(input("Votre âge : "))
          ^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'
```

Pour le dernier essai, avec `abc`, on retrouve la `ValueError` de tout à l'heure. Deux choses à retenir :

- **les deux-points** `:` à la fin de la ligne du `if` et du `else` ;
- **l'indentation** : les lignes décalées de 4 espaces forment le bloc du `if`. C'est le décalage qui dit à Python où le bloc s'arrête. Pas de `fi` : `print("Au revoir.")`, revenu à gauche, s'exécute dans tous les cas.

VS Code décale tout seul après les deux-points, et la touche `Tab` insère 4 espaces. Un décalage oublié, et Python refuse de lancer le programme :

oubli.py

```
age = 20
if age >= 18:
print("Vous êtes majeur.")
```

Le décalage oublié

```
$ python3 oubli.py
  File "/home/alice/python-semaine/oubli.py", line 3
    print("Vous êtes majeur.")
    ^
IndentationError: expected an indented block after 'if' statement on line 2
```

Plusieurs cas avec `elif` (contraction de *else if*) :

mention.py

```
note = float(input("Votre note sur 20 : "))

if note >= 16:
    print("Mention très bien")
elif note >= 14:
    print("Mention bien")
elif note >= 12:
    print("Mention assez bien")
elif note >= 10:
    print("Admis")
else:
    print("Recalé")
```

Trois notes

```
$ python3 mention.py
Votre note sur 20 : 17
Mention très bien
$ python3 mention.py
Votre note sur 20 : 12
Mention assez bien
$ python3 mention.py
Votre note sur 20 : 8
Recalé
```

Python teste les conditions **dans l'ordre** et s'arrête à la première qui est vraie. Avec 17, `note >= 14` est vraie aussi, mais Python n'arrive jamais jusque-là.

Comparer	Python	Combiner	Python
égal	`==`	et	`and`
différent	`!=`	ou	`or`
plus petit, plus grand	`<` `>`	non	`not`
plus petit ou égal, plus grand ou égal	`<=` `>=`	entre deux valeurs	`10 <= note <= 20`

nombre.py

```python
n = int(input("Un nombre : "))

if n == 0:
    print("C'est zéro.")
elif n > 0 and n % 2 == 0:
    print("Positif et pair.")
elif n > 0:
    print("Positif et impair.")
else:
    print("Négatif.")
```

Trois essais

```
$ python3 nombre.py
Un nombre : 7
Positif et impair.
$ python3 nombre.py
Un nombre : -3
Négatif.
$ python3 nombre.py
Un nombre : 0
C'est zéro.
```

> **Piège.** `=` range une valeur dans une variable, `==` compare. Écrire `if n = 0:` provoque une `SyntaxError`, et VS Code le souligne en rouge avant même de lancer.

> **En bash** : `if (( age >= 18 )); then ... else ... fi`. En Python : `if age >= 18:`, un bloc décalé, `else:`, un bloc décalé. Et `elif` s'écrit pareil dans les deux langages.

Exercices du lundi

Un fichier par exercice dans votre dossier `python-semaine` : `l1_bonjour.py`, `l2_calculs.py`… Testez chaque programme avec plusieurs valeurs, y compris des valeurs « bizarres » (0, un nombre négatif).

L1. Bonjour personnalisé NIVEAU 1

Demandez le prénom et la ville de l'utilisateur, puis affichez :
`Bonjour Alice, il fait beau à Angers aujourd'hui ?`

L2. La calculatrice NIVEAU 1

Demandez deux nombres entiers, puis affichez leur somme, leur différence, leur produit, leur quotient (avec `/`), le quotient entier et le reste. Une ligne par opération, sous la forme `17 // 5 = 3`.

L3. Pair ou impair NIVEAU 1

Vous l'avez écrit en bash il y a trois semaines : demandez un nombre et dites s'il est pair ou impair. Comparez les deux versions : laquelle est la plus lisible ?

L4. Celsius en Fahrenheit NIVEAU 1

Demandez une température en degrés Celsius (elle peut être à virgule : `float`), et affichez-la en Fahrenheit, avec une décimale. La formule : `F = C × 9 / 5 + 32`. Vérifiez : 100 °C font 212 °F, et 37,5 °C font 99,5 °F.

L5. Le plus grand de trois NIVEAU 2

Demandez trois nombres et affichez le plus grand, **sans** utiliser la fonction `max` de Python. Testez avec le plus grand en premier, en deuxième, en troisième, et avec deux nombres égaux.

L6. Le prix du cinéma NIVEAU 2

La place coûte 11 €. Moins de 14 ans : 6 €. Étudiants de moins de 26 ans : 8 €. 65 ans et plus : 8 €. Demandez l'âge, puis demandez « Êtes-vous étudiant (o/n) ? » **seulement si c'est utile**, et affichez le prix.

L7. Année bissextile NIVEAU 2

Une année est bissextile si elle est divisible par 4, **sauf** si elle est divisible par 100, **sauf** si elle est divisible par 400. Demandez une année et dites si elle est bissextile. À tester : 2024 (oui), 2026 (non), 1900 (non), 2000 (oui).

L8. Le rendu de monnaie NIVEAU 3

Un client doit payer un montant entier en euros et donne un billet de 50 €. Affichez comment lui rendre la monnaie avec le moins de billets et de pièces possible, parmi 20 €, 10 €, 5 €, 2 € et 1 €. N'affichez que les lignes utiles. Indice : `//` et `%` font tout le travail.

```
Montant à payer : 13
Rendu : 37 €
1 billet(s) de 20 €
1 billet(s) de 10 €
1 billet(s) de 5 €
1 pièce(s) de 2 €
```

Que se passe-t-il si le montant dépasse 50 € ? Gérez ce cas.

Mémo du lundi

Je veux…	En bash	En Python
afficher	`echo "Bonjour"`	`print("Bonjour")`
créer une variable	`nom="Alice"`	`nom = "Alice"`
utiliser une variable	`"$nom"`	`nom`
mélanger texte et variables	`"$nom a $age ans"`	`f"{nom} a {age} ans"`
lire au clavier	`read -p "Âge : " age`	`age = input("Âge : ")`
convertir en nombre	inutile	`int(texte)`, `float(texte)`
calculer	`$(( a * b ))`	`a * b`
division entière, reste	`$(( a / b ))`, `$(( a % b ))`	`a // b`, `a % b`
condition	`if (( a > b )); then … fi`	`if a > b:` + bloc décalé
sinon si	`elif (( … )); then`	`elif …:`
et, ou, non	`&&`, `||`, `!`	`and`, `or`, `not`
lancer le programme	`bash script.sh`	`python3 script.py`